

Obsidian RAG chatbot

#project

ENG 

Table of Contents:

-  **Project Overview**
-  **How RAG Works**
 - 1. Retrieval
 - 2. Generation
-  **System Architecture**
-  **ingest.py**
 -  **Purpose**
 -  **Output**
-  **search.py**
 -  **Purpose**
 -  **How It Works**
 -  **Output**
 -  **Role in the RAG Pipeline**
-  **chatbot.py**
 -  **Purpose**
 -  **How It Works**
 -  **Output + Example**

Project Overview

Obsidian RAG Chatbot is a local AI assistant designed to work with Obsidian notes.

The project implements a Retrieval-Augmented Generation (RAG) architecture that allows the system to retrieve relevant information from notes and use it as context for generating answers with a Large Language Model (LLM).

Unlike traditional keyword-based search, the system uses semantic search, which makes it possible to find relevant information even when different wording is used in the query and the notes.

All computations in this project are performed locally:

- Obsidian serves as the knowledge base;
 - Sentence Transformers generates embeddings;
 - ChromaDB is used as the vector database;
 - LM Studio runs a local LLM;
 - Python integrates all components into a single system.
-

How RAG Works

RAG (Retrieval-Augmented Generation) consists of two main stages:

1. Retrieval

The system finds the most relevant text fragments within the notes.

To achieve this:

- notes are split into chunks;
 - an embedding is generated for each chunk;
 - embeddings are stored in ChromaDB;
 - the user's query is also converted into an embedding;
 - ChromaDB retrieves the most similar chunks.
-

2. Generation

The retrieved chunks are passed to the language model as context.

The LLM uses this context to generate an answer.

As a result, the model responds not only based on its own knowledge but also on information stored in the Obsidian Vault.



System Architecture

```
Obsidian Notes (.md)
    ↓
  ingest.py
    ↓
Sentence Transformers
    ↓
  ChromaDB
    ↓
  search.py
    ↓
 chatbot.py
    ↓
  LM Studio
    ↓
Generated Answer
```



ingest.py



Purpose

The `ingest.py` file is responsible for preparing the knowledge base for the RAG system.

Main responsibilities:

- loading markdown notes from the Obsidian Vault;

- **splitting text into chunks;**
- **generating embeddings;**
- **storing embeddings in ChromaDB.**

```
# Import required modules for document loading,
# text chunking, embeddings, and vector storage
from langchain_community.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
import os

# Path to the Obsidian Vault
VAULT_PATH = "/Users/***/my Obsidian"

# List for storing loaded documents
documents = []

# Traverse all folders and subfolders inside the vault
for root, dirs, files in os.walk(VAULT_PATH):
    for file in files:
        # Process only markdown files
        if file.endswith(".md"):
            # Build full file path
            path = os.path.join(root, file)
            try:
                # Load markdown file
                loader = TextLoader(path, encoding="utf-8")
                # Add loaded document to the collection
                documents.extend(loader.load())
            except Exception as e:
                print(f"Error loading {path}: {e}")

# Display the number of loaded documents
print(f"Loaded {len(documents)} documents")

# Split documents into smaller text chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,          # Maximum chunk size
    chunk_overlap=100        # Overlap between chunks
)

# Generate chunks from loaded documents
chunks = text_splitter.split_documents(documents)
```

```
print(f"Created {len(chunks)} chunks")
# Load embedding model
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
# Create local ChromaDB vector database
db = Chroma.from_documents(
    chunks,                # Document chunks
    embeddings,             # Embedding model
    persist_directory="./chroma_db" # Storage location
)
print("ChromaDB created successfully!")
```



Output

After execution, the script creates a local vector knowledge base (`chroma_db`) that is later used for semantic search and chatbot functionality.



search.py



Purpose

The `search.py` file is responsible for semantic search within the knowledge base.

Its primary objective is to find the most relevant notes for a user's query using embeddings and ChromaDB.

Unlike traditional keyword-based search, semantic search retrieves information based on meaning rather than exact word matches.

```
# Import ChromaDB and embedding model
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings

# Load the same embedding model used during indexing
embeddings = HuggingFaceEmbeddings(
```

```
model_name="sentence-transformers/all-MiniLM-L6-v2"
)
# Connect to the existing ChromaDB database
db = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings
)
# Get user question
query = input("Ask something: ")
# Perform semantic similarity search
results = db.similarity_search(query, k=3)
# Display retrieved chunks and source files
for i, result in enumerate(results):
    print("\n")
    print(f"Result {i+1}")
    print("-" * 50)
    # Print retrieved text chunk
    print(result.page_content)
    print("\nSOURCE:")
    # Print source note path
    print(result.metadata.get("source"))
```

How It Works

1. The user enters a query.
2. The query is converted into an embedding.
3. ChromaDB retrieves the most similar chunks.
4. The system returns the retrieved text fragments.
5. The source note is displayed for each result.

Output

After execution, the user receives:

- relevant text fragments;

- the path to the source note;
 - semantic search results.
-

Role in the RAG Pipeline

Architecturally, the `search.py` file is responsible for the following stage:

```
User Question
      ↓
Embedding
      ↓
Vector Search
      ↓
Relevant Context
```

This stage retrieves the information that will later be used by the chatbot to generate an answer.

`chatbot.py`

Purpose

The `chatbot.py` file is the core component of the RAG system.

It combines:

- semantic search using ChromaDB;
- embeddings;
- a local language model running through LM Studio;
- final answer generation.

Unlike `search.py`, which only retrieves relevant chunks, `chatbot.py` uses the retrieved results as

context for the LLM and generates a complete answer.

```
# Import ChromaDB and embedding model
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
import requests

# Load the same embedding model used during indexing
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# Connect to the existing ChromaDB database
db = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings
)

# Get user question
query = input("Ask something: ")

# Retrieve the most relevant chunks from the vector database
results = db.similarity_search(query, k=3)

# Combine retrieved chunks into a single context block
context = "\n\n".join([r.page_content for r in results])

# Build prompt for the language model
prompt = f"""
Answer the question using ONLY the context below.
Context:
{context}
Question:
{query}
"""

# Send prompt to LM Studio local API
response = requests.post(
    "http://localhost:1234/v1/chat/completions",
    json={
        "model": "google/gemma-4-e4b",
        "messages": [
            {
                "role": "user",
                "content": prompt
            }
        ]
    }
)
```



```

        ],
        "temperature": 0.3
    }
)

# Extract generated answer
answer = response.json()["choices"][0]["message"]["content"]

# Display answer
print("\n")
print("=" * 50)
print("ANSWER:")
print(answer)

print("\n")
print("=" * 50)
print("SOURCES:")

# Display source notes used for retrieval
for r in results:
    print(r.metadata.get("source"))

```

How It Works

1. The user enters a question.
 2. The query is converted into an embedding.
 3. ChromaDB retrieves the 3 most relevant chunks.
 4. All retrieved chunks are combined into a single context.
 5. A prompt is created for the language model.
 6. The prompt is sent to the local LLM through the LM Studio API.
 7. The model generates an answer based on the provided context.
 8. The system displays the generated answer along with the list of source notes.
-



Output + Example

After execution, the user receives:

- an answer generated by the LLM;
- a list of notes used as context sources.

Example:

```
Ask something: what do I know about Random Forest ?
```

```
=====
```

```
ANSWER:
```

```
According to the context, you know that Random Forest is:
```

- * One of the most universal models for real-world business tasks
- * Used in various AI applications:
 - + Regression
 - + Classification
 - + Clustering
 - + Feature engineering
 - + Cross validation
 - + Overfitting

```
=====
```

```
SOURCES:
```

This file transforms the system from a simple semantic search engine into a complete RAG chatbot capable of answering questions using information stored in the Obsidian knowledge base.

UA 

Зміст:

-  Що це за проєкт
-  Як працює RAG
 - 1. Retrieval

- 2. Generation
-  Архітектура системи
-  ingest.py
 -  Призначення
 -  Результат
-  search.py
 -  Призначення
 -  Як працює
 -  Результат
 -  Роль у RAG Pipeline
-  chatbot.py
 -  Призначення
 -  Як працює
 -  Результат + приклад

Що це за проєкт

Obsidian RAG Chatbot — це локальний AI-помічник для роботи з нотатками Obsidian.

Проєкт реалізує архітектуру Retrieval-Augmented Generation (RAG), яка дозволяє знаходити релевантну інформацію серед нотаток та використовувати її як контекст для генерації відповідей за допомогою великої мовної моделі (LLM).

На відміну від звичайного пошуку за ключовими словами, система використовує semantic search, що дозволяє знаходити інформацію навіть тоді, коли в запитанні та нотатках використовуються різні формулювання.

У цьому проєкті всі обчислення виконуються локально:

- Obsidian використовується як база знань;

- Sentence Transformers створює embeddings;
 - ChromaDB використовується як vector database;
 - LM Studio запускає локальну LLM;
 - Python об'єднує всі компоненти в єдину систему.
-

Як працює RAG

RAG (Retrieval-Augmented Generation) складається з двох основних етапів:

1. Retrieval

Система знаходить найбільш релевантні фрагменти тексту серед нотаток.

Для цього:

- нотатки розбиваються на chunks;
 - для кожного chunk створюється embedding;
 - embeddings зберігаються у ChromaDB;
 - запит користувача також перетворюється в embedding;
 - ChromaDB знаходить найбільш схожі chunks.
-

2. Generation

Знайдені chunks передаються до мовної моделі як контекст.

LLM використовує цей контекст для генерації відповіді.

Таким чином модель відповідає не лише на основі власних знань, а й на основі інформації з Obsidian Vault.

Архітектура системи

Obsidian Notes (.md)



ingest.py



Sentence Transformers



ChromaDB



search.py



chatbot.py



LM Studio



Generated Answer



ingest.py



Призначення

Файл `ingest.py` відповідає за підготовку бази знань для RAG-системи.

Основні задачі:

- завантаження markdown-нотаток з Obsidian Vault;
- розбиття тексту на chunks;
- створення embeddings;
- збереження embeddings у ChromaDB.

```
# Import required modules for document loading,  
# text chunking, embeddings, and vector storage  
from langchain_community.document_loaders import TextLoader  
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```

from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
import os
# Path to the Obsidian Vault
VAULT_PATH = "/Users/***/my Obsidian"
# List for storing loaded documents
documents = []
# Traverse all folders and subfolders inside the vault
for root, dirs, files in os.walk(VAULT_PATH):
    for file in files:
        # Process only markdown files
        if file.endswith(".md"):
            # Build full file path
            path = os.path.join(root, file)
            try:
                # Load markdown file
                loader = TextLoader(path, encoding="utf-8")
                # Add loaded document to the collection
                documents.extend(loader.load())
            except Exception as e:
                print(f"Error loading {path}: {e}")
# Display the number of loaded documents
print(f"Loaded {len(documents)} documents")
# Split documents into smaller text chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,          # Maximum chunk size
    chunk_overlap=100        # Overlap between chunks
)
# Generate chunks from loaded documents
chunks = text_splitter.split_documents(documents)
print(f"Created {len(chunks)} chunks")
# Load embedding model
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
# Create local ChromaDB vector database
db = Chroma.from_documents(
    chunks,                  # Document chunks
    embeddings,              # Embedding model
    persist_directory="./chroma_db" # Storage location

```

```
)  
print("ChromaDB created successfully!")
```

Результат

Після запуску `ingest.py` створюється локальна векторна база знань (`chroma_db`), яка використовується для semantic search та роботи чатбота.

search.py

Призначення

Файл `search.py` відповідає за semantic search у базі знань.

Його основне завдання — знайти найбільш релевантні нотатки для запиту користувача за допомогою embeddings та ChromaDB.

На відміну від звичайного пошуку за ключовими словами, semantic search шукає інформацію за змістом, а не за точним збігом слів.

```
# Import ChromaDB and embedding model  
from langchain_community.vectorstores import Chroma  
from langchain_community.embeddings import HuggingFaceEmbeddings  
  
# Load the same embedding model used during indexing  
embeddings = HuggingFaceEmbeddings(  
    model_name="sentence-transformers/all-MiniLM-L6-v2"  
)  
# Connect to the existing ChromaDB database  
db = Chroma(  
    persist_directory="./chroma_db",  
    embedding_function=embeddings  
)  
# Get user question  
query = input("Ask something: ")  
# Perform semantic similarity search  
results = db.similarity_search(query, k=3)  
# Display retrieved chunks and source files
```

```
for i, result in enumerate(results):
    print("\n")
    print(f"Result {i+1}")
    print("-" * 50)
    # Print retrieved text chunk
    print(result.page_content)
    print("\nSOURCE:")
    # Print source note path
    print(result.metadata.get("source"))
```

Як працює

1. Користувач вводить запит.
2. Запит перетворюється на `embedding`.
3. ChromaDB знаходить найбільш схожі `chunks`.
4. Система повертає знайдені фрагменти тексту.
5. Для кожного результату виводиться джерело нотатки.

Результат

Після запуску користувач отримує:

- релевантні фрагменти тексту;
- шлях до вихідної нотатки;
- результати `semantic search`.

Роль у RAG Pipeline

Архітектурно файл `search.py` відповідає за етап:

User Question



Embedding



Vector Search



Relevant Context

Саме цей етап дозволяє знайти інформацію, яка буде використана чатботом для генерації відповіді.

chatbot.py

Призначення

Файл `chatbot.py` є основним компонентом RAG-системи.

Він об'єднує:

- semantic search через ChromaDB;
- embeddings;
- локальну мовну модель через LM Studio;
- генерацію фінальної відповіді.

На відміну від `search.py`, який лише знаходить релевантні chunks, `chatbot.py` використовує знайдені результати як контекст для LLM та генерує повноцінну відповідь.

```
# Import ChromaDB and embedding model
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
import requests

# Load the same embedding model used during indexing
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
```

```

)
# Connect to the existing ChromaDB database
db = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings
)
# Get user question
query = input("Ask something: ")
# Retrieve the most relevant chunks from the vector database
results = db.similarity_search(query, k=3)
# Combine retrieved chunks into a single context block
context = "\n\n".join([r.page_content for r in results])
# Build prompt for the language model
prompt = f"""
Answer the question using ONLY the context below.
Context:
{context}
Question:
{query}
"""
# Send prompt to LM Studio local API
response = requests.post(
    "http://localhost:1234/v1/chat/completions",
    json={
        "model": "google/gemma-4-e4b",
        "messages": [
            {
                "role": "user",
                "content": prompt
            }
        ],
        "temperature": 0.3
    }
)

# Extract generated answer
answer = response.json()["choices"][0]["message"]["content"]

# Display answer
print("\n")

```

```
print("=" * 50)
print("ANSWER:")
print(answer)

print("\n")
print("=" * 50)
print("SOURCES:")

# Display source notes used for retrieval
for r in results:
    print(r.metadata.get("source"))
```

Як працює

1. Користувач вводить запитання.
2. Запит перетворюється на `embedding`.
3. ChromaDB знаходить 3 найбільш релевантні `chunks`.
4. Усі знайдені `chunks` об'єднуються в єдиний контекст.
5. Створюється `prompt` для мовної моделі.
6. `Prompt` передається до локальної LLM через LM Studio API.
7. Модель генерує відповідь на основі отриманого контексту.
8. Система виводить відповідь та список використаних джерел.

Результат + приклад

Після запуску користувач отримує:

- відповідь, згенеровану LLM;
- список нотаток, які були використані як джерела контексту.

Приклад:

Ask something: what do I know about Random Forest ?

=====

ANSWER:

According to the context, you know that **Random Forest** is:

- * One of the most universal models for real-world business tasks
- * Used in various AI applications:
 - + Regression
 - + Classification
 - + Clustering
 - + Feature engineering
 - + Cross validation
 - + Overfitting

=====

SOURCES:

Саме цей файл перетворює систему з простого semantic search на повноцінний RAG chatbot, здатний відповідати на запитання на основі власної бази знань Obsidian.